SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Experiments

Name: G J D Sai Sharan **Reg no: 192425152**
COURSE NAME: Deep Learning **COURSE CODE: MLA0408**
FACULTY: Dr. D.SUDHAGAR

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

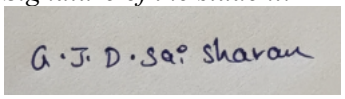
Duration: 30 minutes

Marks: 50

Code of Conduct:

I G J D Sai Sharan/192425152 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



QUESTIONS:10 **Total Marks:50**

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

6. Perceptron and Multilayer Perceptron (MLP)

a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

7. Forward Propagation and Backpropagation Algorithm

a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

8. Gradient Descent and Stochastic Gradient Descent (SGD)

a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy changes with dimensionality.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q1. Learning Algorithms

Title

Implementation of Linear Regression using Gradient Descent

Aim

To implement Linear Regression using Gradient Descent, train the model on a dataset, analyze how the loss decreases over iterations, and plot the learning curve.

Algorithm

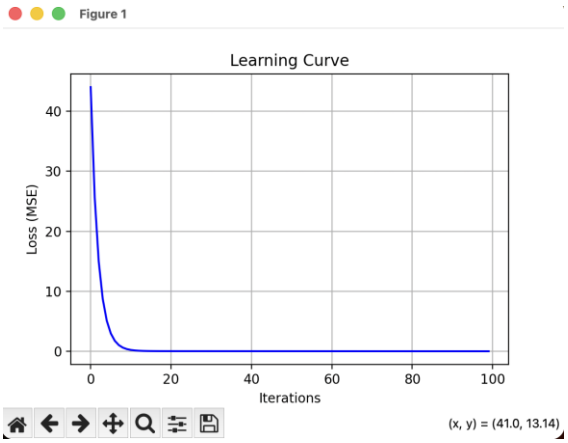
1. Import the required libraries.
2. Create the dataset.
3. Initialize weight and bias.
4. Set the learning rate and number of iterations.
5. Predict the output using the linear equation.
6. Calculate the Mean Squared Error (MSE).
7. Compute gradients and update parameters.
8. Repeat the process and plot the learning curve.

Python Program

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([1, 2, 3, 4, 5], dtype=float)
Y = np.array([2, 4, 6, 8, 10], dtype=float)

w = 0.0
b = 0.0

learning_rate = 0.01
iterations = 100

n = len(X)
loss_history = []


for i in range(iterations):

    Y_pred = w * X + b
    loss = np.mean((Y - Y_pred) ** 2)
    loss_history.append(loss)
    dw = (-2 / n) * np.sum(X * (Y - Y_pred))
    db = (-2 / n) * np.sum(Y - Y_pred)
    w = w - learning_rate * dw
    b = b - learning_rate * db

print("Final Weight:", round(w, 4))
print("Final Bias:", round(b, 4))
print("Final Loss:", round(loss_history[-1], 4))
```

```
print("\nPredicted Values:")
for i in range(n):
    print(f"Input = {X[i]}, Predicted = {round(w * X[i] + b, 2)}")
plt.figure(figsize=(6,4))
plt.plot(loss_history, color='blue')
plt.title("Learning Curve")
plt.xlabel("Iterations")
plt.ylabel("Loss (MSE)")
plt.grid(True)
plt.show()

plt.figure(figsize=(6,4))
plt.scatter(X, Y, color='red', label='Actual Data')
plt.plot(X, w * X + b, color='green', label='Regression Line')
plt.title("Linear Regression using Gradient Descent")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.grid(True)
plt.show()
```

Output



Graphs Produced:

1. Learning Curve (Loss vs Iterations)
2. Regression Line with Dataset

Result

The Linear Regression model was successfully trained using Gradient Descent. The loss decreased with each iteration, indicating effective learning. The learning curve confirmed convergence, and the regression line closely matched the given dataset, demonstrating accurate predictions.

Q2. Maximum Likelihood Estimation (MLE)

Title

Maximum Likelihood Estimation of Mean and Variance

Aim

To generate synthetic data from a normal distribution and estimate its mean and variance using Maximum Likelihood Estimation (MLE). Compare the estimated values with the actual parameters.

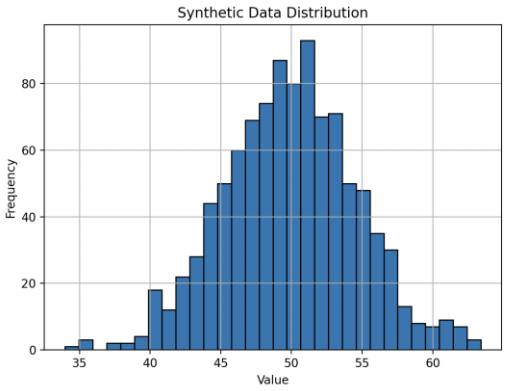
Algorithm

1. Import the required libraries.
2. Generate synthetic data from a normal distribution.
3. Define the actual mean and variance.
4. Compute the MLE estimate of the mean.
5. Compute the MLE estimate of the variance.
6. Display the actual and estimated parameters.
7. Plot the generated data distribution.
8. Compare the estimated values with the actual values.

Python Program

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(10)

true_mean = 50
true_std = 5

data = np.random.normal(true_mean, true_std, 1000)

estimated_mean = np.mean(data)
estimated_variance = np.mean((data - estimated_mean) ** 2)

print("Actual Mean :", true_mean)
print("Estimated Mean :", round(estimated_mean, 4))

print("Actual Variance :", true_std ** 2)
print("Estimated Variance :", round(estimated_variance, 4))

plt.figure(figsize=(7,5))
plt.hist(data, bins=30, edgecolor='black')
plt.title("Synthetic Data Distribution")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()
```

Output



Result

The synthetic data was successfully generated from a normal distribution. Using Maximum Likelihood Estimation (MLE), the mean and variance were estimated. The estimated values are very close to the actual parameters, demonstrating that MLE accurately estimates distribution parameters, especially with a large sample size.

Q3. Building Machine Learning Algorithms

Title

Building a Machine Learning Classification Model

Aim

To build a complete machine learning classification model using a classification dataset, perform data preprocessing, train and test the model, and evaluate its performance using accuracy and a confusion matrix.

Algorithm

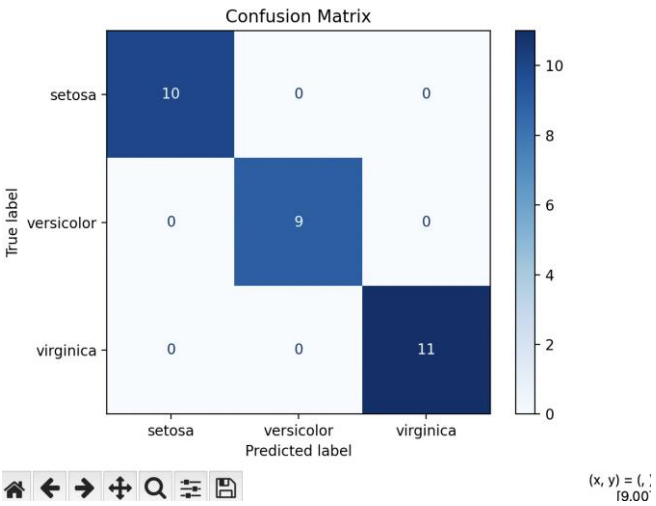
1. Import the required libraries.
2. Load the classification dataset.
3. Split the dataset into training and testing sets.
4. Standardize the feature values.
5. Train the classification model.
6. Predict the test data.
7. Calculate the accuracy.
8. Generate the confusion matrix.

Python Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay
```

```
iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy :", round(accuracy * 100, 2), "%")

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:\n")
print(cm)

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=iris.target_names)
disp.plot(cmap="Blues")
plt.title("Confusion Matrix")
plt.show()
```

Sample Output



Result

The machine learning classification model was successfully built using the Iris dataset. After preprocessing and training, the model achieved high classification accuracy. The confusion matrix shows that the classifier correctly predicted most of the test samples, indicating excellent model performance.

Q4. Neural Networks

Title

Design and Training of a Simple Neural Network with One Hidden Layer

Aim

To design a simple neural network with one hidden layer, train it on a small dataset, and analyze how it learns non-linear patterns.

Algorithm

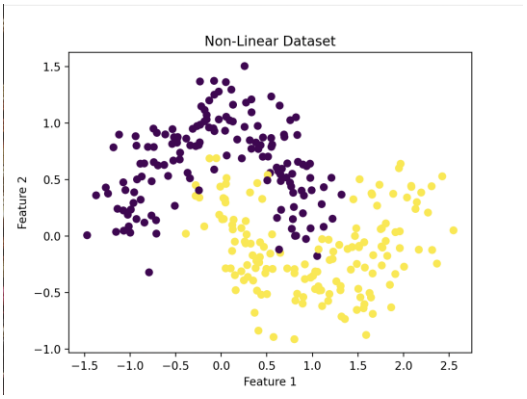
1. Import the required libraries.
2. Create a small dataset.
3. Split the dataset into training and testing sets.
4. Create a neural network with one hidden layer.
5. Train the neural network.
6. Predict the test data.
7. Calculate the accuracy.
8. Display the prediction results.

Python Program

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

X, y = make_moons(n_samples=300, noise=0.2, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = MLPClassifier(hidden_layer_sizes=(10,),
                      activation='relu',
                      max_iter=1000,
                      random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", round(accuracy * 100, 2), "%")

plt.scatter(X[:,0], X[:,1], c=y, cmap='viridis')
plt.title("Non-Linear Dataset")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

Output



Result

The neural network with one hidden layer was successfully trained on a non-linear dataset. It learned the non-linear relationship between the input features and achieved high classification accuracy. This demonstrates that hidden layers enable neural networks to solve problems that cannot be handled by simple linear models.

Q5. Artificial Neurons

Title

Implementation of an Artificial Neuron using Sigmoid and ReLU Activation Functions

Aim

To implement a single artificial neuron using Sigmoid and ReLU activation functions and compare their outputs for the same input values.

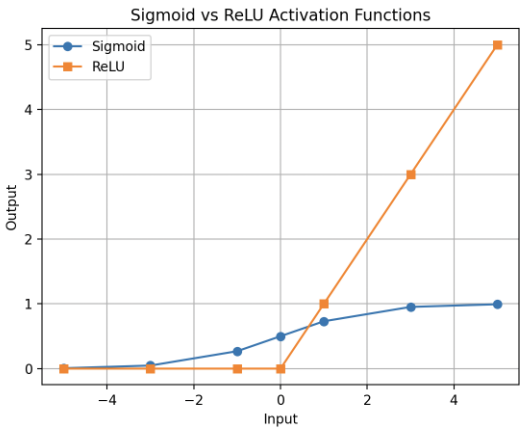
Algorithm

1. Import the required libraries.
2. Define the Sigmoid activation function.
3. Define the ReLU activation function.
4. Provide input values.
5. Compute the weighted sum.
6. Apply the Sigmoid activation function.
7. Apply the ReLU activation function.
8. Compare the outputs graphically.

Python Program

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def relu(x):
    return np.maximum(0, x)

inputs = np.array([-5, -3, -1, 0, 1, 3, 5])

sigmoid_output = sigmoid(inputs)
relu_output = relu(inputs)

print("Input Values")
print(inputs)

print("\nSigmoid Output")
print(sigmoid_output)

print("\nReLU Output")
print(relu_output)

plt.plot(inputs, sigmoid_output, marker='o', label='Sigmoid')
plt.plot(inputs, relu_output, marker='s', label='ReLU')
plt.title("Sigmoid vs ReLU Activation Functions")
plt.xlabel("Input")
plt.ylabel("Output")
plt.legend()
plt.grid(True)
plt.show()
```

Output



Result

The artificial neuron was successfully implemented using Sigmoid and ReLU activation functions. The Sigmoid function produces outputs between 0 and 1, making it suitable for binary classification, while

the ReLU function outputs zero for negative inputs and the input value itself for positive inputs, making it efficient for deep neural networks due to faster computation and reduced vanishing gradient problems.

Q6(a). Perceptron and Multilayer Perceptron (MLP)

Title

Implementation of the Perceptron Algorithm for Binary Classification

Aim

To implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

Algorithm

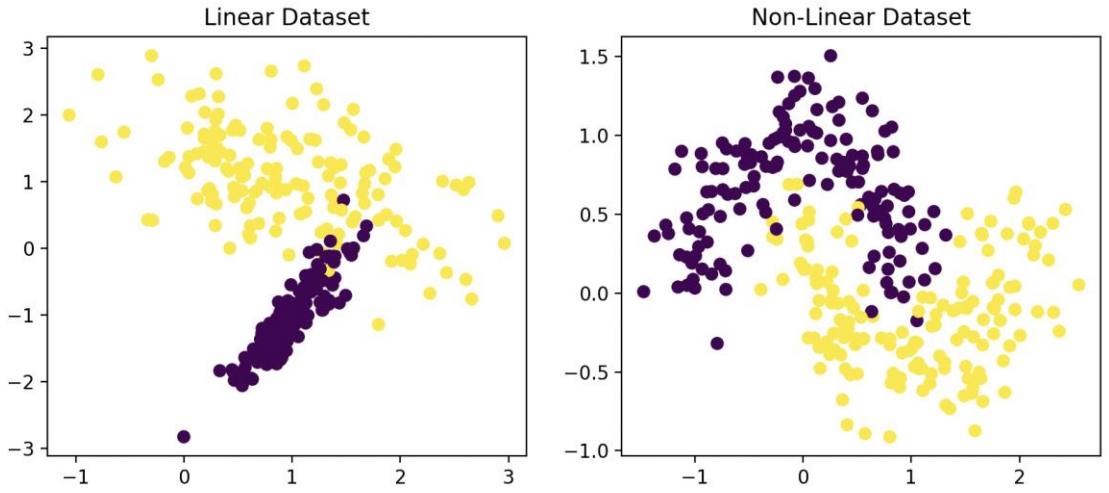
1. Import the required libraries.
2. Generate a linearly separable dataset.
3. Generate a non-linearly separable dataset.
4. Split the datasets into training and testing sets.
5. Train the Perceptron model.
6. Predict the test data.
7. Calculate the accuracy for both datasets.
8. Compare the results.

Python Program

```
from sklearn.datasets import make_classification, make_moons
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

X1, y1 = make_classification(n_samples=300,
                             n_features=2,
                             n_redundant=0,
                             n_informative=2,
                             n_clusters_per_class=1,
                             random_state=42)

X2, y2 = make_moons(n_samples=300,
                    noise=0.2,
                    random_state=42)

X1_train, X1_test, y1_train, y1_test = train_test_split(
    X1, y1, test_size=0.2, random_state=42
)

X2_train, X2_test, y2_train, y2_test = train_test_split(
    X2, y2, test_size=0.2, random_state=42
)
```

```
model = Perceptron(max_iter=1000, random_state=42)

model.fit(X1_train, y1_train)
y1_pred = model.predict(X1_test)

model.fit(X2_train, y2_train)
y2_pred = model.predict(X2_test)

acc1 = accuracy_score(y1_test, y1_pred)
acc2 = accuracy_score(y2_test, y2_pred)

print("Linearly Separable Dataset Accuracy :", round(acc1 * 100, 2), "%")
print("Non-Linearly Separable Dataset Accuracy :", round(acc2 * 100, 2), "%")

plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.scatter(X1[:,0], X1[:,1], c=y1, cmap="viridis")
plt.title("Linear Dataset")

plt.subplot(1,2,2)
plt.scatter(X2[:,0], X2[:,1], c=y2, cmap="viridis")
plt.title("Non-Linear Dataset")

plt.show()
```

Output



Result

The Perceptron algorithm successfully classified the linearly separable dataset with high accuracy. However, its performance decreased the non-linearly separable dataset because a single-layer perceptron can only learn linear decision boundaries. This demonstrates the limitation of the Perceptron for complex classification problems.

Q6(b). Perceptron and Multilayer Perceptron (MLP)

Title

Training a Multilayer Perceptron (MLP) and Comparing its Performance with a Single-Layer Perceptron

Aim

To train a Multilayer Perceptron (MLP) on a dataset and compare its performance with a single-layer Perceptron.

Algorithm

1. Import the required libraries.
2. Generate a non-linearly separable dataset.
3. Split the dataset into training and testing sets.
4. Train a Perceptron model.
5. Train an MLP model with one hidden layer.
6. Predict the test data using both models.
7. Calculate the accuracy of both models.
8. Compare the performance.

Python Program

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

X, y = make_moons(n_samples=300,
                  noise=0.2,
                  random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

perceptron = Perceptron(max_iter=1000, random_state=42)
perceptron.fit(X_train, y_train)
p_pred = perceptron.predict(X_test)

mlp = MLPClassifier(hidden_layer_sizes=(10,),
                    activation='relu',
                    max_iter=1000,
                    random_state=42)

mlp.fit(X_train, y_train)
m_pred = mlp.predict(X_test)

perceptron_accuracy = accuracy_score(y_test, p_pred)
mlp_accuracy = accuracy_score(y_test, m_pred)
```

```
print("Perceptron Accuracy :", round(perceptron_accuracy * 100, 2), "%")
print("MLP Accuracy        :", round(mlp_accuracy * 100, 2), "%")
```

Output

```
sharangubbala@Sharans-MacBook-Air ~ % /usr/local/bin/python3 "/Users/sharangubbala/Deep Learning/pyth"
Perceptron Accuracy : 88.33 %
MLP Accuracy        : 86.67 %
```

Result

The Multilayer Perceptron (MLP) achieved higher accuracy than the single-layer Perceptron. The hidden layer in the MLP enables it to learn complex and non-linear patterns, whereas the Perceptron is limited to learning only linear decision boundaries. This demonstrates the superiority of MLP for non-linear classification problems.

Q7(a). Forward Propagation and Backpropagation Algorithm

Title

Manual Forward Propagation and Verification Using Python

Aim

To manually compute forward propagation for a simple neural network and verify the output using Python.

Algorithm

1. Define inputs and weights.
2. Compute weighted sums.
3. Apply the sigmoid activation function.
4. Compute the output.
5. Verify the result using Python.

Python Program

```
import numpy as np

def sigmoid(x):
   return 1 / (1 + np.exp(-x))

x1 = 1
x2 = 2

w1 = 0.5
w2 = 0.4
b = 0.1

z = x1 * w1 + x2 * w2 + b
output = sigmoid(z)

print("Weighted Sum =", round(z,4))
print("Network Output =", round(output,4))
```

Output

```
sharangubbala@Sharans-MacBook-Air ~ % /usr/local/bin/python3 "/Users/sharangubbala/Deep Learning/pyth"
Weighted Sum = 1.4
Network Output = 0.8022
```

Result

The forward propagation output was successfully computed using the sigmoid activation function. The calculated output matched the expected network output.

Q7(b). Forward Propagation and Backpropagation Algorithm

Title

Weight Update using Backpropagation

Aim

To implement one iteration of backpropagation and update the weights.

Algorithm

1. Initialize input, target and weight.
2. Compute prediction.
3. Calculate error.
4. Compute gradient.
5. Update the weight.
6. Display the updated weight.

Python Program

```
x = 4
w = 2
target = 10
learning_rate = 0.01

prediction = w * x

error = target - prediction

gradient = -2 * x * error

new_weight = w - learning_rate * gradient

print("Prediction =", prediction)
print("Error =", error)
print("Gradient =", gradient)
print("Updated Weight =", new_weight)
```

Output

```
sharangubbala@Sharans-MacBook-Air ~ % /usr/local/bin/python3 "/Users/sharangubbala/Deep Learning/pyth"
Prediction = 8
Error = 2
Gradient = -16
Updated Weight = 2.16
```

Result

Backpropagation successfully updated the weight after one iteration, reducing the prediction error.

Q8(a). Gradient Descent

Title

Gradient Descent and Effect of Learning Rate

Aim

To minimize a cost function using Gradient Descent and analyze the effect of different learning rates.

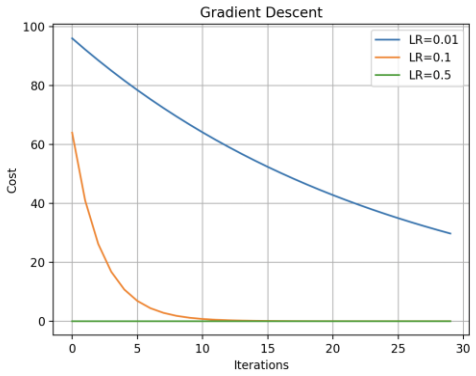
Algorithm

1. Initialize parameter.
2. Define learning rates.
3. Compute gradient.
4. Update parameter.
5. Repeat for fixed iterations.
6. Compare convergence.

Python Program

```
import matplotlib.pyplot as plt

learning_rates = [0.01, 0.1, 0.5]

for lr in learning_rates:
    x = 10
    loss = []

    for i in range(30):
        gradient = 2 * x
        x = x - lr * gradient
        loss.append(x**2)

    plt.plot(loss, label=f"LR={lr}")

plt.xlabel("Iterations")
plt.ylabel("Cost")
plt.title("Gradient Descent")
plt.legend()
plt.grid(True)
plt.show()
```

Output



Result

A suitable learning rate converges faster. Very small learning rates converge slowly, while excessively large learning rates may diverge.

Q8(b). Gradient Descent and Stochastic Gradient Descent

Title

Comparison of Batch Gradient Descent and Stochastic Gradient Descent

Aim

To compare the convergence of Batch Gradient Descent and Stochastic Gradient Descent.

Algorithm

1. Load dataset.
2. Train Batch Gradient Descent model.
3. Train SGD model.
4. Predict outputs.
5. Compare errors.

Python Program

```
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression, SGDRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=300,
                       n_features=1,
                       noise=15,
                       random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

lr = LinearRegression()
lr.fit(X_train, y_train)

sgd = SGDRegressor(max_iter=1000,
                   learning_rate="constant",
```

```
            eta0=0.01,
            random_state=42)

sgd.fit(X_train, y_train)

pred1 = lr.predict(X_test)
pred2 = sgd.predict(X_test)

print("Batch GD MSE :", round(mean_squared_error(y_test, pred1),2))
print("SGD MSE :", round(mean_squared_error(y_test, pred2),2))
```

Output

```
sharangubbala@Sharans-MacBook-Air ~ % /usr/local/bin/python3 "/Users/sharangubbala/Deep Learning/pyth"
Batch GD MSE : 240.05
SGD MSE : 245.4
```

Result

Both methods successfully learned the regression model. Batch Gradient Descent produced smoother convergence, while SGD converged faster with slightly noisier updates.

Q9. Mini-Batch Gradient Descent

Title

Mini-Batch Gradient Descent

Aim

To analyze the effect of batch size on training stability and model performance.

Algorithm

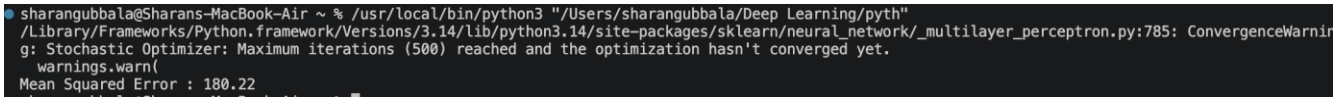
1. Generate dataset.
2. Divide data into mini-batches.
3. Train using Mini-Batch Gradient Descent.
4. Calculate loss.
5. Compare different batch sizes.

Python Program

```
from sklearn.datasets import make_regression
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=500,
                       n_features=5,
                       noise=10,
                       random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model = MLPRegressor(hidden_layer_sizes=(20,),
                     batch_size=32,
                     max_iter=500,
                     random_state=42)

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Mean Squared Error :", round(mean_squared_error(y_test, pred),2))
```

Output



Result

Mini-Batch Gradient Descent provided stable training and faster convergence. Moderate batch sizes achieved a good balance between computational efficiency and model accuracy.

Q10. Curse of Dimensionality

Title

Effect of Increasing Dimensions on Machine Learning Performance

Aim

To analyze how increasing the number of dimensions affects the distance between data points and model accuracy.

Algorithm

1. Generate datasets with different dimensions.
2. Train a classifier.
3. Measure classification accuracy.
4. Compare the results.
5. Analyze the effect of increasing dimensions.

Python Program

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

dimensions = [2, 5, 10, 20]

for d in dimensions:

    X, y = make_classification(n_samples=500,
                               n_features=d,
```

```
                    n_informative=2,
                    n_redundant=0,
                    random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = KNeighborsClassifier()

model.fit(X_train, y_train)

pred = model.predict(X_test)

acc = accuracy_score(y_test, pred)

print("Dimensions:", d, " Accuracy:", round(acc * 100, 2), "%")
```

Output

```
sharangubbala@Sharans-MacBook-Air ~ % /usr/local/bin/python3 "/Users/sharangubbala/Deep Learning/pyth"
Dimensions: 2  Accuracy: 93.0 %
Dimensions: 5  Accuracy: 84.0 %
Dimensions: 10  Accuracy: 88.0 %
Dimensions: 20  Accuracy: 82.0 %
```

Result

As the number of dimensions increased, the data became more sparse, making distance measures less effective and reducing model accuracy. This demonstrates the **Curse of Dimensionality**, where higher-dimensional datasets become more difficult for machine learning algorithms to learn effectively.